UNIVERSITY COLLEGE OF ENGINEERING

End Semester Examination

Object Oriented Programming with Java (BCS403)

Roll No. _____

Subject: Object Oriented Programming with Java

Time: 3 Hours

Max. Marks: 70

Code: BCS403

General Instructions:

1. Attempt all questions as per the rules specified in each section.
2. The use of a programmable calculator is prohibited.
3. Assume suitable data if any missing.

SECTION A — SECTION A

Short Answer Questions

2 marks each | Attempt all questions

Q.No.	Question	Marks	CO
1	In a security-sensitive application, you need to ensure that the class 'MasterKey' can only be extended by 'AdminKey' and 'UserKey' within the same module. Which keyword introduced in recent Java versions would you use, and how does it provide more flexibility than the 'final' keyword?	**2**	**CO1**
2	A developer is manually closing a 'FileInputStream' in a finally block, but the close() method itself throws an IOException which masks the original exception from the try block. Propose a modern Java construct that manages this 'suppressed exception' logic automatically.	**2**	**CO2**
3	Consider the lambda expression: (var x, var y) -> x.process(y). Under what specific circumstances is it mandatory or beneficial to use 'var' for lambda parameters instead of omitting the types entirely?	**2**	**CO3**
4	You are required to store a collection of unique IP addresses that must be retrieved in the exact order they were first detected. Which specific implementation of the 'Set' interface would you choose, and what is its internal data structure overhead compared to a HashSet?	**2**	**CO4**
5	In a Spring Boot REST controller, you have a method that should return a raw JSON string directly to the web client. Explain why using @RestController is more architecturally concise than using @Controller combined with @ResponseBody on every method.	**2**	**CO5**
6	A class 'Config' contains 50 static constants. If you use 'import static com.app.Config.*', analyze the potential impact on code readability versus the performance impact on the JVM during class loading.	**2**	**CO1**

Q.No.	Question	Marks	CO
7	Explain the 'JVM Reaction' when an unchecked exception like NullPointerException occurs in a thread that does not have a try-catch block. What happens to the other active threads in the same process?	**2**	**CO2**

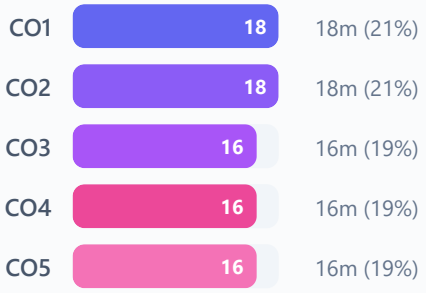
SECTION B — SECTION B			
Attempt any 3 questions.			
7 marks each | Attempt all questions			
Q.No.	**Question**	**Marks**	**CO**
8	Design a 'PaymentGateway' system using Interfaces and Abstract Classes. The system must support 'CreditCard' and 'UPI' payments. All payments must have a 'transactionID', but only 'CreditCard' payments require 'CVV' validation. Demonstrate how you would use polymorphism to process a list of different payment types while ensuring that validation logic remains specific to the implementation.	**7**	**CO1**
9	A multi-threaded server uses a 'SharedResource' where multiple 'Reader' threads can access data simultaneously, but a 'Writer' thread must have exclusive access. Implement a synchronization mechanism using 'wait()' and 'notifyAll()' that prevents 'Thread Starvation' for the writer while maintaining high throughput for the readers.	**7**	**CO2**
10	You are given a 'List' where each Transaction has an 'amount', 'category', and 'date'. Use the Java Stream API to: (i) Filter transactions above 10,000, (ii) Group them by category, and (iii) Calculate the total sum for each category. Explain how the 'Intermediate' and 'Terminal' operations in your code facilitate 'Lazy Evaluation'.	**7**	**CO3**
11	A logistics company needs to sort 'Package' objects. Each package has a 'weight' and a 'priority' (High, Medium, Low). Implement a custom sorting logic using the 'Comparator' interface such that packages are sorted primarily by priority (High first) and secondarily by weight (descending) for packages with the same priority level.	**7**	**CO4**
12	In a Spring Boot e-commerce application, a 'DiscountService' bean is a Singleton, but it depends on a 'UserSession' bean which must be unique for every HTTP request. Explain the 'Scope Impedance' problem that arises here and provide a solution using Spring's Scoped Proxy or ObjectFactory approach.	**7**	**CO5**

SECTION C — SECTION C			
Detailed Analytical Questions			
7 marks each | Attempt all questions			
Q.No.	**Question**	**Marks**	**CO**
13	**(a)** Analyze the 'Diamond Problem' in the context of Multiple Inheritance. How does Java's support for 'Default Methods' in Interfaces re-introduce this ambiguity, and how does the compiler force the developer to resolve it?	**7**	**CO1**
	OR		
	(b) Compare 'Records' with traditional 'Classes' for representing immutable data. Explain why Records do not allow instance fields outside the header and how the 'Compact Constructor' simplifies validation compared to standard class constructors.	**7**	**CO1**
14	**(a)** You are designing a file processing system. Differentiate between using 'Byte Streams' and 'Character Streams' when handling a file that contains UTF-16 encoded text. What are the consequences of using a 'FileInputStream' to read multi-byte characters?	**7**	**CO2**
	OR		
	(b) Develop a scenario where a 'Checked Exception' is thrown by a method in a low-level library. Explain the architectural trade-offs between catching and wrapping it into a 'User-Defined Unchecked Exception' versus using the 'throws' keyword to propagate it to the UI layer.	**7**	**CO2**
15	**(a)** Explain the concept of 'Functional Interfaces' and their relationship with Lambda Expressions. How does the target typing mechanism in the Java compiler determine which functional interface a lambda expression belongs to?	**7**	**CO3**
	OR		
	(b) The Java Module System (Project Jigsaw) was introduced to solve 'JAR Hell'. Describe how the 'module-info.java' file uses 'exports' and 'requires' to achieve strong encapsulation and reliable configuration that was previously missing in the Classpath approach.	**7**	**CO3**
16	**(a)** Analyze the internal performance of 'HashMap' when a poorly designed hashCode() function causes many 'collisions'. How did Java 8 improve the worst-case time complexity of HashMap lookups from O(n) to O(log n)?	**7**	**CO4**
	OR		
	(b) A navigation system requires storing a map of 'City' objects where the keys must always be sorted by their GPS coordinates. Compare the suitability of 'LinkedHashMap' versus 'TreeMap' for this requirement, focusing on insertion cost and memory footprint.	**7**	**CO4**
17	**(a)** Contrast the use of @PathVariable and @RequestParam in a RESTful API. Design two different URL patterns for a 'Search' feature and a 'Resource Detail'	**7**	**CO5**

Q.No.	Question	Marks	CO
	feature, justifying which annotation is better for each based on SEO and API readability.		
	OR		
	(b) Describe the Spring Bean Lifecycle. If you need to perform a custom cleanup (like closing a database connection) before the application shuts down, which lifecycle callback annotation would you use, and how does Spring ensure this is called?	**7**	**CO5**

Question Paper Analysis

Bloom's Taxonomy Distribution

- K1 – Remember: 0m (0%)
- K2 – Understand: 11m (13%)
- K3 – Apply: 9m (11%)
- K4 – Analyze: 29m (35%)
- K5 – Evaluate: 28m (33%)
- K6 – Create: 7m (8%)

Course Outcome Distribution